Execution Divergence Graphs: Effective Discovery of Control-Flows from Execution Traces as Fuzzing Feedback

Anonymous Authors

No Institute Given

Abstract. Fuzz testing is a popular approach to the security testing of proprietary software. Efficient testing strategies rely on execution feedback to guide the input generation process, particularly when the basic blocks in the binary can be directly observed and instrumented. Unfortunately, collecting such feedback is impossible in scenarios such as in-situ fuzzing of black-box devices and the fuzzing of obfuscated compiled binaries. In this work, we discuss approaches to guide the fuzzer using feedback derived from a control-flow-graph-like (CFG-like) structure constructed from runtime execution.
We start by outlining a simple divergence-detection approach that identifies unique execution traces, and then present an improved approach based on an Execution Divergence Graph (EDG). We implement both approaches and demonstrate that they outperform a baseline blind fuzzer. In addition, we discuss particular challenges, such as repeated code execution in loops, and show that the EDG-based approach handles them effectively. We then demonstrate that our approach enables effective fuzzing of a number of obfuscated targets, and compare its performance in scenarios where static instrumentation is impossible. While we focus on a scenario in which full instruction traces are directly observable by the attacker, our scheme can also be applied in scenarios with other feedback channels, such as power consumption.

1 Introduction

Fuzzing is an automated software-testing technology that works by injecting random inputs (called "fuzz" [1]) into a program to expose bugs, crashes, or security vulnerabilities. Unguided fuzzers generate inputs from a corpus [2, 3] or predefined-input rules [4] to explore simple programs. However, unguided fuzzers struggle to reach deeper code paths in complex programs, where serious vulnerabilities are often located. To address this issue, modern guided test approaches [5–10] use coverage, which represents how many code paths are visited during testing, to guide the fuzzer. This coverage-guided fuzzing requires static instrumentation to generate feedback.

Instrumentation, however, has several limitations. First, it requires access to the target's source code or binary for successful code modification. Even when

a binary is available, its author can use tools to obfuscate it, rendering instrumentation ineffective. For instance, obfuscators such as Movfuscator [11] and Tigress [12] can hide all standard branch instructions. Without distinct control-flow instructions like branches, the concept of a basic block is lost, and the fuzzer cannot accurately map or track connections between basic blocks (edges). Consequently, the fuzzer cannot obtain the crucial control-flow information, such as which paths an input has visited, needed to guide its exploration.

In this work, we present several approaches to address this issue, and show how to generate suitable feedback for fuzzing by analyzing target execution traces. Such traces could, for example, consist of a sequence of the executed instruction addresses without further annotation. Even without static analysis of the target binary, we show how to process such execution traces to detect execution of novel code segments, while maintaining computational and memory efficiency. While naive approaches to this problem simply store prior execution traces and compare them to new traces, we show that such approaches are particularly unable to detect repeated execution of code segments, and thus vastly overestimate the novelty of inputs. To address this, we introduce the concept of Execution Divergence Graphs (EDGs), a CFG-like structure for tracking observed execution sequences and describing the target program. Using an EDG to track input executions allows valuable coverage feedback to be generated for standard fuzzers. In our experiments, we demonstrate that our approach achieves performance similar to classic instrumentation-guided fuzzing, without requiring explicit instrumentation of the target binary.

Our contributions are as follows:

- We demonstrate that the divergence between execution traces can guide a coverage-guided fuzzer.
- We introduce a graph structure based on execution traces to improve the execution-divergence feedback given to the fuzzer.
- We implement our approach and evaluate it on several real-world programs to demonstrate that our approach is applicable to standard fuzzing tasks.
- We also demonstrate our approach on several obfuscated targets (where classic instrumentation is impossible), and show that our approach can still guide the fuzzer to explore the obfuscated programs efficiently.
- Our artifact can be found at `https://anonymous.4open.science/r/EDG-BBC4`.

2 Background

2.1 Coverage-Guided Fuzzing

The ultimate objective of a coverage-guided fuzzer is to generate inputs such that, by the end of a fuzzing campaign, every possible control-flow path has been traversed at least once. Coverage-guided fuzzers usually use instrumentation to extract the control flow of the target for a given input. Based on this feedback,

they attempt to generate additional inputs that exercise different paths from those taken by previous inputs, thus maximizing code coverage.

Coverage is often measured in the form of *edge coverage* [5]. The target binary is instrumented so that it emits an edge-coverage hash map for each input. This map expresses the transitions between basic blocks that occur at runtime in a condensed form. The fuzzer maintains a cumulative hash map to track all known edges and their frequencies of occurrence. Comparing the hash map of a single execution run (using one particular input) with the cumulative hash map allows the fuzzer to assess whether new edges were discovered or whether known edges were traversed more or less frequently.

2.2 Guided Black-box Fuzzing

Although coverage-guided fuzzing effectively finds bugs in programs that support post-instrumentation edge feedback, it is inapplicable in various scenarios, such as network and protocol fuzzing, where static instrumentation is not feasible. Therefore, black-box fuzzing that does not leverage the internal information of a program is widely applied in these scenarios.

However, conventional black-box fuzzing approaches, such as mutation-based and generation-based fuzzing, are not sufficient in these scenarios. Thus, these approaches design their own feedback mechanism based on the externally observable states such as responses from IoT devices, protocol states, and messages between network nodes [13–15] to *guide* black-box fuzzing.

3 Novelty Detection from Execution Traces

3.1 System Model

We describe the components illustrated in Figure 1 of the system model.

Black-box Target is a system or device that accepts input for execution. A major challenge arises when the binary is inaccessible. This prevents third-party testers from directly recovering the underlying code, which is a prerequisite for effective software testing, especially coverage-guided fuzzing.

Observer is an external device (e.g., J-Trace Debugger or an oscilloscope) or software (e.g., GDB) that captures the *raw signals* from a black-box target. The observer decodes the raw signals into the *execution traces*, which record the time-series information flow during program execution. The data points vary by observer type: if the observer is a debugging tool, the trace records text indicating which instruction or function was executed at a specific timestamp. Alternatively, if the observer is a measurement device, such as an oscilloscope monitoring power signals, the time-series data consists of floating-point values representing power consumption.

Execution Divergence Analysis Component is designed to identify differences between execution traces from a pair of program executions. It executes the *Divergence Detection* process to quantify the difference between two execution traces (T_1 and T_2), using an element-wise heuristic function $H(T_1, T_2) = v$,

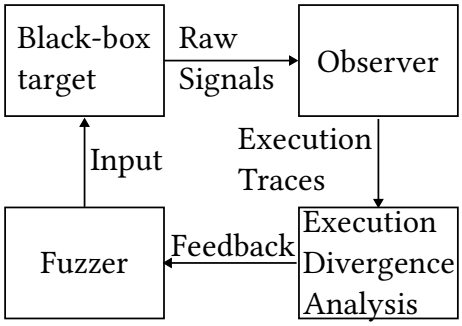


Fig. 1: System Overview

which yields a difference vector v aligned in time with the traces. A divergence point is then designated at any timestamp s where the corresponding value v_s exceeds a predefined threshold τ ($v_s > \tau$), indicating the time at which the observed divergence between the traces begins. The component then leverages the divergence results to create *trace segments*. If the trace segments are sequential instructions, we refer to these trace segments as *code segments*. **Fuzzer** uses the trace segments as feedback from the black-box target and generates inputs accordingly.

The Fuzzing Loop. The pseudocode of the fuzzing loop is shown in Algorithm 1. The fuzzer first generates input i from its given corpus without feedback (line 2). The generated input i is then sent to the black-box target connected to the observer for processing (line 4). When the processing is complete, the observer obtains the trace information t for the execution divergence analysis module (line 5). Subsequently, the module returns the analysis result as feedback to the fuzzer, which then generates the next input accordingly (line 6). If the given edge feedback is deemed interesting by the fuzzer, the input i is added to the corpus $\mathbb{C}$.

Algorithm 1: Overall fuzzing loop with execution divergence feedback.

Input: $\mathbb{C}$: the seeded corpus for the fuzzer
Data: $\mathbb{T}$: previously observed unique traces; i: input; f: feedback provided by $DivergenceAnalysis(.)$; t: the observed execution trace.

1 $\mathbb{T} \leftarrow \emptyset$;
2 $i, \mathbb{C} \leftarrow Fuzzer(\emptyset, \mathbb{C})$; `// Generate an initial input without feedback.`
3 **while** *true* **do**
4 $\quad t \leftarrow Observer(Blackbox(i))$;
5 $\quad f, \mathbb{T} \leftarrow DivergenceAnalysis(t, \mathbb{T})$;
6 $\quad i, \mathbb{C} \leftarrow Fuzzer(f, \mathbb{C})$;

3.2 Research Questions

- **RQ1.** Given a new execution trace and all previous traces, how can we efficiently determine whether the new execution trace follows a previously known code path (and identify the matching old trace(s))?
- **RQ2.** Given a new execution trace and all previous traces, how can we efficiently determine whether the new trace contains novel code segments or novel transitions?
- **RQ3.** How can novel code and transition discovery be used as feedback for the fuzzer?

3.3 Simple Divergence Approach

We start by addressing **RQ1**. The simplest approach for detecting novelty in a new execution trace relies on a divergence-detection function that compares the new trace against all previously recorded traces (line 2 in Alg 2).

We first calculate the vector v using the heuristic function $H(.)$ (line 3) and check at which index the value v_s exceeds the threshold τ. This indicates that two traces start to diverge at s, and the comparison then continues with the next T (line 6-7 and line 2). When the new trace t has been compared with all traces in $\mathbb{T}$, we add the trace into the trace set $\mathbb{T}$ (line 9). Lastly, we return the t and $\mathbb{T}$ to the Fuzzer (line 10).

In contrast, if there is no value higher than threshold, the new trace t is a duplicate and we stop further comparison with the rest of the traces in $\mathbb{T}$. The matching trace T and the trace set $\mathbb{T}$ are then returned to the fuzzer (line 8). This method deems a new execution trace to be novel only if it diverges from every single recorded trace.

Although simple to implement, this technique suffers from two performance-related drawbacks: **Recomparing Shared Traces** and the **False Novelty from Repeating Code Segments**.

The Recomparing Shared Traces issue arises because the method redundantly checks common code segments shared among multiple recorded execution traces. Assuming that all n recorded traces share the same code segment `ABCDE` in the beginning, the approach has to perform at least $n \times 5$ identical comparisons. This excessive and redundant computation drastically degrades performance, particularly as the number of the recorded traces grows.

False Novelty from Repeating Code Segments arises when code segments repeated through loop iterations are mistakenly classified as novel. For instance, a program contains a code segment `C` that checks whether a byte is legal or not. The program uses this code segment to validate input bytes, which may vary in number depending on input length. When an input length is 3 bytes, the code segment will repeat 3 times, creating execution trace `CCC`. If a new input is 5 bytes long, the new execution trace will become `CCCCC`. By comparing the two execution traces `CCC` and `CCCCC`, the approach concludes that the trace `CCCCC` is novel. However, this code segment has already been discovered.

Algorithm 2: Pseudocode of simple Divergence Analysis Approach.

```
Input: 𝕋: set of unique traces; t: the new trace; τ threshold for trace segment
       comparison
Result: 𝕋 and t
Data: duplicate: a boolean value to denote a new trace is identical to one of
      the traces in 𝕋.
duplicate ← false;
for T in 𝕋 do
    v ← H(T, t);
    for v_s in v                                  // Check if two traces differ
    do
        if v_s > τ then
            break;
    return t, 𝕋
𝕋 ← 𝕋 ⋃ {t}
return t, 𝕋
```

3.4 Execution Divergence Graph-based Approach

To address **RQ2** and overcome the significant performance impact caused by the redundant recomparison of shared traces, we now introduce the Execution Divergence Graph (*EDG*)-based approach. The EDG data structure enables merging identical code segments shared across different execution traces. Unlike the baseline approach, which stores each trace identified as divergent, the EDG approach merges common, identical code segments between new and existing traces before storage. For instance, given a new trace `ABCDEFG` and an existing trace `ABCDECCA` within the EDG, the divergence detection function first identifies the shared code segments `ABCDE`. The graph then factors out this shared segment and connects it to the remaining segments, `FG` and `CCA`. In this example, the newly created shared code segment `ABCDE` and the two remaining segments are novel. Eventually, even if the EDG contains n different execution traces sharing the code segment `ABCDE`, any new incoming trace only needs to compare with that merged segment once, drastically improving efficiency.

Repeated Code Segments. However, the simple EDG cannot solve the false-novelty problems caused by repeating code segments. Consider a simple EDG initialized with a stored execution trace T_1=`ABAB`. When a new incoming execution trace T_{new}=`ABABAB` arrives, it triggers the graph to create a new trace T_2=`AB`, connected to T_1. The graph then marks T_2 as a novel code segment. Furthermore, when there is a new execution trace T_{new}=`ABABABAB`, the graph iteratively creates another code segment T_3=`AB`, connected to T_2 and again marks T_3 as novel. In fact, T_2 and T_3 are the repeating code segments `AB`, demonstrating the simple EDG's failure to recognize repeating code segments.

Backlinking in EDG. Backlinking is a mechanism we introduced to resolve the false positives from repeating code segments in the graph. This process is initiated immediately when the divergence detection function identifies a diver-

gence between the incoming trace and the stored trace, signaling a candidate new segment. Backlinking then broadcasts the candidate segments across the entire graph and searches the entire graph for any existing identical code segments. Following the previous example, when the graph obtains T_2=`AB`, backlinking broadcasts T_2 and discovers that there is an exact match with its parent trace T_1=`ABAB`, leading to its merger with the identical part in T_1 and establishing a *self-loop connection*. After refactoring the identical part, T_1 is reduced to `AB` and broadcast by backlinking. Another exact match then occurs between T_1 and T_2, causing T_1 to be merged once again with T_2. With backlinking, no matter how many times a new incoming trace repeats `AB`, the EDG always considers it a non-novel code segment.

Execution Divergence Analysis with EDG. The new trace t is first used to traverse EDG $\mathbb{T}$, which stores all previously observed traces (line 1 in Alg 3). After traversal, EDG returns the edges f traversed by the new trace and the unmatched residual part t'. If the trace has a perfect match in the graph, then t' is empty, and the traversed edges f are returned. Conversely, when t' is not empty, the EDG triggers $backlink(.)$ to integrate t' (line 3). Since the graph is updated after $backlink(.)$, and some nodes and edges change, we traverse the trace t again to obtain the latest traversed edges (line 4). More details about $traverse(.)$ and $backlink(.)$ are provided in Section 4.3 and Section 4.4, respectively.

Algorithm 3: Pseudocode of *EDG*-based Execution Divergence Analysis.

Input: $\mathbb{T}$: EDG, containing unique traces observed so far; t is a new trace.
Result: f: the traversed edges in EDG.
Data: t': the unmatched part of the new trace t after the first $traverse(.)$.

```
1 (t', f) ← traverse(𝕋, t);
2 if t' is not ∅ then
3 |   𝕋 ← backlink(𝕋, t');
4 |   (∅, f) ← traverse(𝕋, t);
5 return f, 𝕋
```

3.5 Generating Feedback from Execution Divergence Analysis

Overview. In this section, we address **RQ3** and discuss *how, based on an EDG, we can guide an AFL fuzzer by execution traces while fuzzing an uninstrumented target*. We first explain how AFL's hit-count bucket mechanism determines whether feedback is interesting and how *simple-div* and *EDG* interact with AFL through feedback. We use five execution traces shown in Table 1 as examples to describe how *simple-div* and *EDG* work together with AFL.

AFL's Hit-Count Bucket. AFL does not preserve the exact execution count of each edge. Instead, it maps the count into a small set of hit-count buckets, such as 1, 2, 3–4, 5–8, 9–16, 17–32, 33–128, and 129 or more. This

Table 1: An example demonstrates how *simple-div* and *EDG* convert execution traces into AFL feedback.

Trace	*simple-div*		*EDG*		
	Feedback	Interesting	Segments	Feedback	Interesting
S	S:1	Yes	S	∅:1	Yes
SAB	SAB:1	Yes	S;AB	S→AB:1	Yes
SABABABAB	SABABABAB:1	Yes	S;AB	S→AB:1,AB→AB:3,	Yes
SABABABAB	SABABABAB:1	No	S;AB	S→AB:1,AB→AB:3,	No
SABABABABAB	SABABABABAB:1	Yes	S;AB	S→AB:1,AB→AB:4	No

coarse-grained encoding reduces noise from minor count fluctuations while still capturing meaningful execution changes. Therefore, an input is considered interesting not only when it reaches a previously unseen edge, but also when it drives an already known edge into a bucket that has not been observed before.

Feedback from *simple-div*. For each trace, *simple-div* generates one edge feedback for AFL, always with a hit count of 1. Each individual trace is mapped to a different edge, while identical traces are mapped to the same edge. In our example, four edges are reported to AFL, corresponding to S, SAB, SABABABAB, and SABABABABAB. As the second occurrence of SABABABAB is identical to the first occurrence, it is reported as the same edge, again with a hit count of 1. Thus, AFL does not consider it interesting. In summary, AFL considers all distinct traces in this example interesting because they are reported as previously unseen edges, whereas the repeated occurrence of SABABABAB is ignored because it is reported in the same way as before. Based on the five example traces, we identify two explicit drawbacks of *simple-div*: first, every divergent trace is treated similarly because all divergent traces are placed in bucket 1; second, the repeated segment AB causes repeated traces to be treated as novel simply because it appears more times.

Feedback from *EDG* Analysis. Unlike *simple-div*, *EDG* converts traces into a graph representation. In this graph, links between nodes (i.e., transitions between code blocks) are used as feedback. This is very similar to standard AFL feedback based on links in a CFG. The first trace is a special case for *EDG* because the graph contains only a single node (S). The feedback reported to AFL is represented as a special edge 0 with a count of 1. When the second trace, SAB, arrives, our *EDG* analysis adds the segment AB and the corresponding link between S and AB into the EDG. This link is then reported to AFL with a count of 1. The *EDG* analysis does not find any new code segment in the third trace, SABABABAB, but it finds that the transition AB→AB repeats three times in the trace. Since the transition AB→AB is new and occurs three times in the execution, AFL places the new transition in bucket 3-4 and marks it as interesting. The fourth trace is identical to the third trace and is therefore not interesting. For the fifth trace, our *EDG* analysis finds that the transition AB→AB repeats four times, which is different from before. However, it remains in bucket 3-4 because

the number of repetitions does not exceed the bucket cap of 4 in this case. Therefore, the fifth trace is not considered interesting by AFL. In summary, this example demonstrates that our *EDG* analysis provides more informative feedback than the simple divergence detection approach.

4 Detailed Design of Execution Divergence Graphs

4.1 Definition

Unlike a control-flow graph, which captures all possible connections between basic blocks based on complete information available from an instrumented binary, the EDG is constructed solely from execution traces collected at runtime from a black-box target. It organizes these traces according to their observed divergences, enabling third-party testers to analyze the target's behavior without access to its internal code structure. Because the EDG depends entirely on observed executions, it cannot reveal execution paths that never occur at runtime, even though such paths may be represented in a traditional control-flow graph. However, the EDG can capture concrete execution paths that static CFG recovery may miss, such as indirect function calls whose callees cannot be determined at compile time. Consequently, the EDG serves as a *dynamic representation of a control-flow graph*, capturing the behavior that actually manifests during execution.

4.2 Graph Components

In the EDG, nodes represent execution segments discovered during the target's runtime (except for the root node), and directed edges connect these segments to describe transitions between execution segments. Because the edges are directed, an edge from Node `X` to Node `Y` implies a parent–child relation, meaning that during execution the target executes `X` first and then proceeds to `Y`. As shown in step (a) of Figure 2, there are three non-root nodes corresponding to the execution segments `A`, `BBZ`, and `CZ`. Starting from the root, the only outgoing edge leads to Node `A`, indicating that `A` is the first execution segment observed at runtime. Node `A` has two outgoing edges to its children, Node `BBZ` and Node `CZ`, illustrating the two possible execution paths that follow segment `A`. Both `BBZ` and `CZ` have no outgoing edges, meaning they represent terminal segments. By following the edges starting from the root, the full set of execution paths occurring during runtime can be reconstructed.

4.3 Match First Traversal

Our *Match First Traversal* algorithm defines how an incoming trace traverses the simple EDG. At each step, the algorithm evaluates all adjacent nodes to determine the next node to explore based on the length of the common prefix. For example, when the trace `ACBBBDZ` is processed by the EDG (Figure 2, step

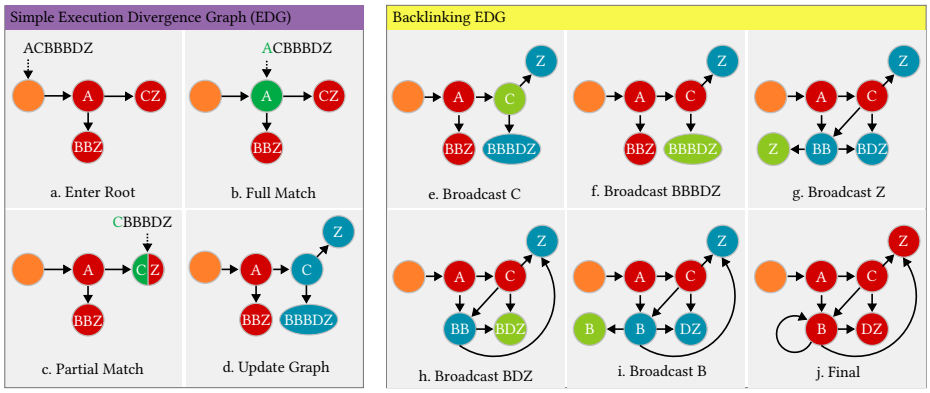


Fig. 2: This illustration contrasts how the Simple and Backlinking Execution Divergence Graphs (EDGs) integrate the new trace `ACBBBDZ`: when a partial match is found (node `CZ`, step c), the Simple EDG immediately appends the remainder of the trace (step d), while the Backlinking EDG broadcasts all new segments (`C`, `Z`, `BBBDZ`) across the graph to check for identical segments. The orange node is the root, dark green nodes signify a full match, half dark-green/red nodes (e.g., `C/Z`) denote a partial match, and blue nodes represent newly created nodes; the light green nodes in Backlinking EDG indicate the node is currently broadcast.

a), it begins at the root and matches Node `A`. Because Node `A` fully matches the first part of the trace, this is a full match, meaning that Node `A` becomes the next node to be explored and the graph remains unchanged (step b). After the match, the algorithm consumes `A` from the trace, resulting in `CBBBDZ`, and proceeds to the next step. In step c, the algorithm finds that `CBBBDZ` and Node `CZ` share a common `C` but diverge afterward. This partial match causes the EDG to factor out the shared segment `C` as a new node, with the remaining segment `Z` becoming another new node connected as a child of `C`. The unmatched rest of the trace, `BBBDZ`, also diverges after `C` and is therefore added as another child of the new node `C` (step d).

4.4 Backlinking Candidates

While the simple EDG directly updates the graph by adding the discovered new segments `C`, `Z`, and `BBBDZ` (step d in Figure 2), the Backlinking EDG instead broadcasts these new segments across the entire graph for additional matching. As shown in step e, the graph first broadcasts Node `C` but finds no match. It then continues with Node `BBBDZ` and discovers that Nodes `BBZ` and `BBBDZ` share the common trace segment `BB` (step f). The graph then factors out this shared segment as a new node `BB`, which is connected to two new nodes corresponding to the divergent parts `BDZ` and `Z`. Node `Z`, which originates from Node `BBZ`, is broadcast next, as shown in step g, where the graph finds that it perfectly

matches another Node `Z`. These two Nodes `Z` are then merged into a single Node `Z`, which inherits the parents `BB` and Node `C` (step h). After broadcasting Node `Z`, the graph broadcasts Node `BDZ` and discovers that Node `BDZ` and Node `BB` share the common trace segment `B`. Consequently, a new Node `B` is created and connected to two new children, Node `DZ` and Node `B`. The graph then applies backlinking to the new child Node `B` in step i and finds a node with an identical trace segment, causing the two Nodes `B` to merge. Because these two Nodes `B` were connected by an edge before merging, the graph creates a *self-loop edge* for the new Node `B`. Once step i is completed, three nodes remain in the broadcast queue: `B`, `Z`, and `DZ`; however, none of them matches any other node in the graph, so no further updates occur and the backlinking process ends (step j). Comparing the results of the simple EDG (step d) and the backlinking EDG (step j), the backlinking version yields fewer redundant trace segments in the final graph.

5 Case Study

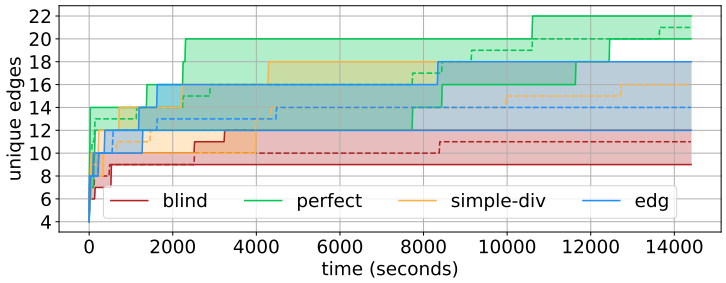
In this section, we first use a simple password checker program as a demonstration of how different levels of feedback affect fuzzing performance. We show how the simple divergence approach easily generates a high number of false positives when the password checker includes an input validator. Hence, we introduce the Execution Divergence Graph to solve this problem. Lastly, we fuzz obfuscated password checkers to demonstrate how obfuscation affects the feedback available to the fuzzer.

5.1 Fuzzing Scenarios

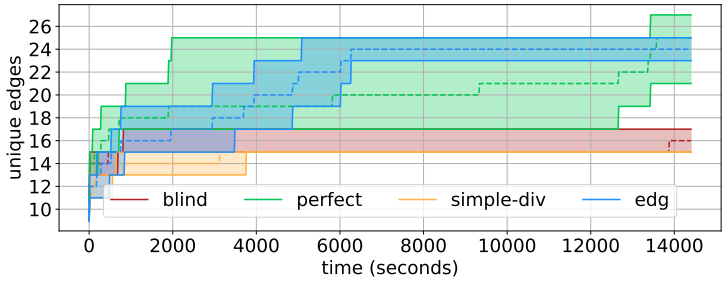
We consider four main fuzzing scenarios: *blind*, *perfect*, *simple-div*, and *EDG*. In the *blind* scenario, the fuzzer receives no feedback from the target. This is the baseline scenario when the target cannot be instrumented. The *perfect* scenario provides ground-truth edge information to the fuzzer. We use this setting as a reference for fuzzer performance under ideal conditions. The *simple-div* (Section 3.3) and *EDG* (Section 3.4) scenarios provide feedback derived from execution traces. For a fuzzer guided by the *simple-div* mechanism, every trace that has not been seen before is treated as a new discovery. In contrast, in the *EDG*-based approach, all collected traces are used to reconstruct a graph that provides more fine-grained trace information, indicating which segments of an incoming trace have already been observed and which are new.

5.2 Password Checker

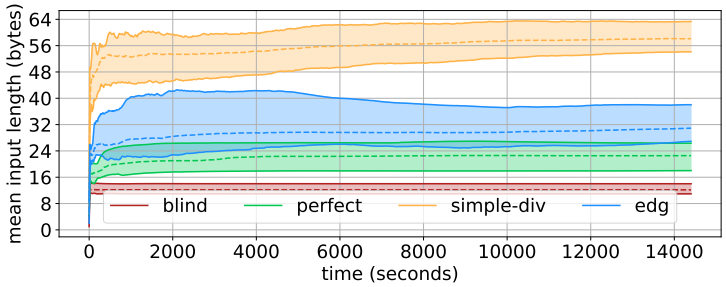
We implement a password checker with secret `badfuzz!` (Alg 4) and run an AFL fuzzer across the four scenarios. Under each scenario, we repeat the fuzzing process five times. Each fuzzing run lasts 4 hours because the fuzzer under the *perfect* scenario already recovers the password. As Figure 3a shows, the fuzzer under the *perfect* scenario discovers the highest number of unique edges (22).



(a) Password checker (Case 1).



(b) Password checker with input validator (Case 2).



(c) Average length of inputs over time (Case 2).

Fig. 3: Figure 3a and Figure 3b demonstrate the number of unique edges discovered over time for the two password checkers across four fuzzing scenarios for 4 hours: *blind*, *perfect*, *simple-div*, and *EDG*. The feedback provided by the divergence-detection-based approaches, *simple-div* and *EDG*, can effectively guide the fuzzer, since both outperform the *blind* scenario, in which the fuzzer receives no feedback (Figure 3a). However, *simple-div* performs poorly in Figure 3b because of repeated basic blocks introduced by the input validator, namely the loop that checks input bytes one by one for null values. These repeated basic blocks are mistakenly treated as divergences and mislead the fuzzer into generating longer inputs instead of focusing on interesting bytes related to the password (Figure 3c).

Algorithm 4: Pseudo code of the password checker.

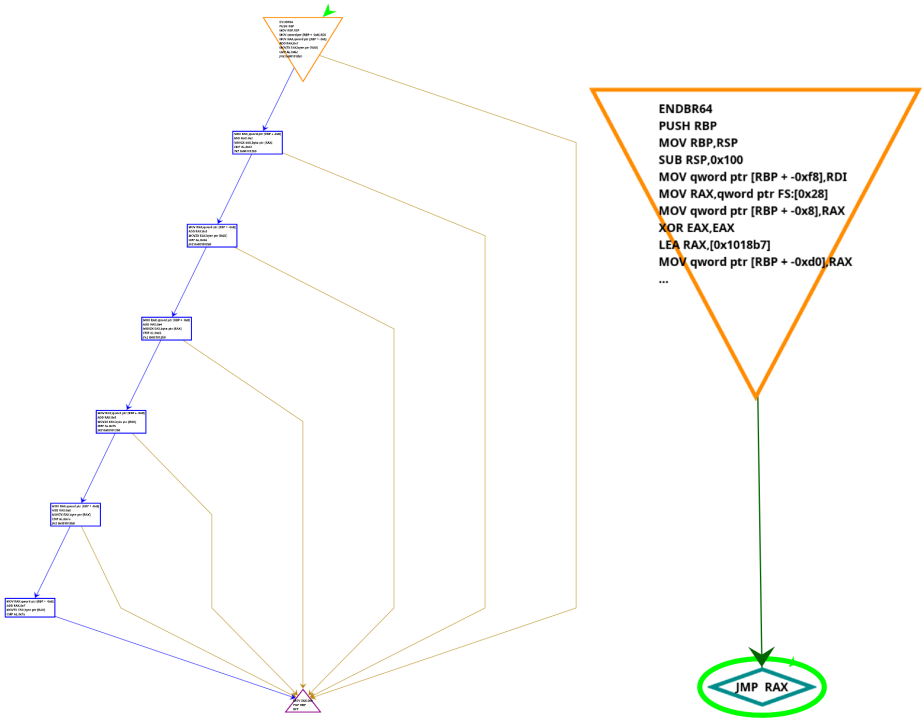
```
   Data: data_ptr: input buffer
 1 if data_ptr[0] = 'b' then
 2 |  if data_ptr[1] = 'a' then
 3 |  |  if data_ptr[2] = 'd' then
 4 |  |  |  if data_ptr[3] = 'f' then
 5 |  |  |  |  if data_ptr[4] = 'u' then
 6 |  |  |  |  |  if data_ptr[5] = 'z' then
 7 |  |  |  |  |  |  if data_ptr[6] = 'z' then
 8 |  |  |  |  |  |  |  if data_ptr[7] = '!' then
 9 |  |  |  |  |  |  |  |  accept password;
10 return 0;
```

The fuzzer without feedback (the *blind* scenario) performs the worst among all scenarios and finds only 12 edges in the best case. When the fuzzer uses trace segments as feedback (*simple-div* and *EDG*), it performs better than in the no-feedback setting. Since all execution paths are related to the characters in the password, the new trace segments found by *simple-div* and *EDG* are identical. Hence, the fuzzer shows comparable performance under these two scenarios.

5.3 Password Checker + Input Validator

However, as we discussed in Section 3.3, the simple divergence approach only detects whether the full trace has been observed before. Any small difference, e.g., if a code segment repeats more times than in previous traces, causes the simple divergence approach to classify the trace as novel, which is a false positive. To demonstrate this issue, we add an input validator to the previous target. The input validator processes the input string and checks each byte for a null value. When the program encounters a null value, it terminates immediately.

Due to this construction, the number of iterations through the input validator depends on the content and length of the input. This causes traces generated from different input lengths to be regarded as novel by the simple divergence approach. As expected, we observe a substantial performance impact in the *simple-div* scenario, as Figure 3b shows: the fuzzer discovers only a very limited number of unique edges. In addition, *simple-div* leads the fuzzer to generate longer inputs than in other scenarios (Figure 3c) because of these input-length-related false positives. As a result, the password checker takes more time to process long inputs than short ones. Because of the longer processing time, the fuzzer generates fewer inputs during a fixed fuzzing duration (*EDG*: 1.6M inputs vs *simple-div*: 1.2M inputs). The *EDG* scenario, on the other hand, is still able to reach deeper parts of the program. These results show that the backlinking mechanism in *EDG* (Section 3.4) provides a substantial improvement in fuzzing performance.



(a) The CFG without obfuscation. (b) The CFG with obfuscation.

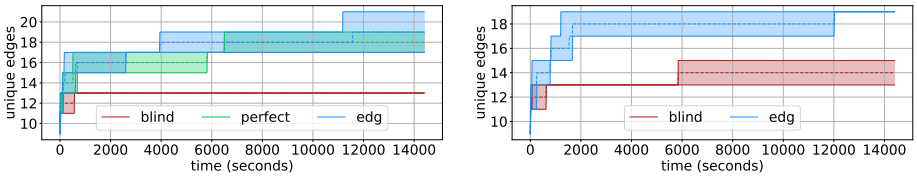
Fig. 4: The comparison between the CFGs of the normal and the obfuscated binaries. The binary is the password checker mentioned in Section 5.2. Before the obfuscation, the CFG shows explicit links between basic blocks and the true/-false paths (Figure 4a). The obfuscation turns the branch instructions into jump instructions, whose destination addresses can only be obtained after execution. Therefore, these jump addresses cannot be found by static instrumentation tool before the program starts (`JMP RAX` in Figure 4b).

5.4 Obfuscated Password Checker

We discuss the obfuscated case in which *EDG* is a suitable replacement for conventional edge feedback for an AFL fuzzer, particularly in cases where static instrumentation is infeasible.

Password Checker's CFG. The CFG of the non-obfuscated password checker is shown in Figure 4a. The figure shows that the password checker has a nested-`if` structure, in which the execution of each basic block depends on the execution of the previous one, except for the entry and exit points.

Control-flow Flattening. Control-flow flattening [16] is one of the most common techniques for obfuscating binaries. It transforms common control structures, such as `if` statements, loops, and function calls, into a central dispatcher that determines which basic block to execute based on its current state. Through



(a) Obfuscated with Control-flow Flatten. (b) Obfuscated with Indirect Jump.

Fig. 5: The number of edges discovered while fuzzing the password checker under two different obfuscation schemes is shown for control-flow flattening (Figure 5a) and indirect jumps (Figure 5b). We obtain these edge counts from the non-obfuscated binary: we first fuzz the obfuscated binaries and then trace the execution of the non-obfuscated binary using the inputs generated during fuzzing. Because both obfuscation schemes significantly modify the original CFG, we observe degraded fuzzing performance compared with Figure 3a and Figure 3b. However, *EDG* and *perfect* still outperform the *blind* fuzzer in the control-flow-flattening case. In contrast, *perfect* no longer works in the indirect-jump case because the control-flow targets are resolved only during execution and cannot be determined at compile time. In this case, the *perfect* fuzzer is effectively reduced to a *blind* fuzzer. Given access to runtime execution traces, *EDG*, in contrast, can still guide the fuzzer by identifying divergences between executions.

control-flow flattening, the password checker's nested-`if` structure is completely flattened and transformed into multiple cases within a `switch` structure. For example, when the non-obfuscated password checker processes the input `bad`, it directly traverses the first `if` statement (`data_ptr[0] == 'b'`) up to the third one (`data_ptr[2] == 'd'`). In contrast, the control-flow-flattened version first enters the dispatcher and is directed to the case corresponding to `data_ptr[0] == 'b'`. The program then updates the dispatcher's state and returns to the dispatcher again. Based on the updated state, the dispatcher transfers execution to the next case, `data_ptr[1] == 'a'`. This example shows that additional dispatcher blocks are introduced into the obfuscated binary. These extra blocks complicate fuzzing and therefore degrade performance in the 4-hour fuzzing experiment (Figure 5a) compared with the non-obfuscated binary (Figure 3b). The mean number of discovered unique edges decreases from 24.75 to 18 for *perfect* and to 19 for *EDG*. With longer fuzzing time, the fuzzer can still recover the whole password, since it can still obtain edge feedback from the instrumented binary.

Indirect Jump. To make the code more obfuscated and harder to instrument automatically, the obfuscator leverages *indirect jump* [17] to replace branch instructions. Instead of transferring execution to an explicit destination (e.g., the address of a basic block), an indirect jump computes the destination at runtime, typically from a register or a memory slot. As a result, the successor basic block is no longer explicit in the instruction itself, which obscures the control-flow graph. In practice, the obfuscator first splits the original code into multiple ba-

sic blocks, assigns each block a corresponding entry in a jump table, and then uses a state variable to select the next target dynamically. Because the branch target is resolved only during execution, analysis and static instrumentation tools have a harder time recovering the original execution order and accurately identifying branch boundaries. Without instrumentation, the original coverage-guided fuzzer becomes blind, since it cannot obtain any edge feedback during fuzzing. However, *EDG* can still guide the fuzzer because it detects changes in execution by comparing traces at runtime. Figure 5b demonstrates that the fuzzer guided by *EDG* feedback reaches deeper parts of the program than the unguided fuzzer.

6 Evaluation

In this section, we first evaluate how efficiently our divergence-detection-based approaches can guide the fuzzer when testing non-obfuscated binaries (Section 6.2). Next, we obfuscate some binaries to hide their control flow in a jump table, which cannot be statically instrumented. We then evaluate whether our approaches can still provide edge feedback that enables the fuzzer to test the obfuscated binaries (Section 6.3).

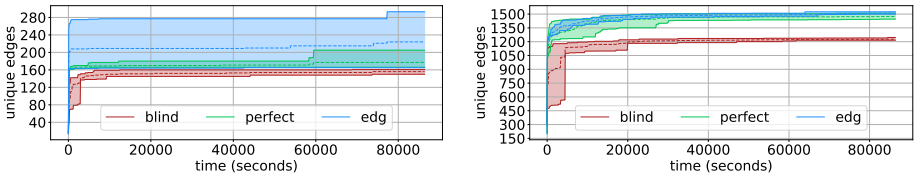
6.1 Settings

Fuzzer. We use LibAFL [6] for fuzzing our non-obfuscated and obfuscated binaries. The number of edges in `EDGE_MAP` is 262,144, which defines the maximum number of edges that can have their own unique IDs. If a binary has more than 262,144 edges, some edges will share the same ID. In terms of feedback, we use `MaxMapPow2Feedback` to enable hit-count buckets. We choose `PowerSchedule` [18] from the default schedulers, since the scheduler prioritizes inputs that trigger uncommon edges. Hence, the scheduler encourages the fuzzer to explore more untouched code in the binaries.

Instruction Tracing. We leverage DynamoRIO [19] to trace instructions during runtime and store the traced instructions in shared memory, where our divergence-detection-based approaches can access them.

Obfuscation. Tigress [12] is a source-to-source obfuscator that provides various obfuscation options for researchers. We use it to convert C code into obfuscated code and compile the result with GCC.

Scenarios. As in Section 5.1, we evaluate three fuzzing scenarios: *blind*, *perfect*, and *EDG*. In Section 6.2, we use the *blind* scenario as the worst-case setting, in which the fuzzer receives no internal feedback. In Section 6.3, the *blind* scenario represents the case in which static instrumentation fails because of the indirect-jump implementation in the obfuscated binaries. We evaluate the *perfect* scenario only in Section 6.2, because the fuzzer cannot access basic-block information without static instrumentation. Hence, in Section 6.3, it is replaced by the *blind* scenario. The *EDG* scenario is our divergence-detection-based fuzzing setting, which has no false-positive problem compared to the *simple-div* approach (Section 5.3).



(a) Results for minmea.

(b) Results for libpng.

Fig. 6: Fuzzing results after 24 hours for fuzzers under three scenarios *blind*, *perfect*, and *EDG*. The dashed line represents the mean number of discovered edges across five fuzzing runs, while the other two solid lines represent the maximum and minimum values, respectively. The results demonstrate that the divergence-detection-based approach (*EDG*) achieves performance comparable to that of the fuzzer with ground-truth edge feedback (*perfect*). More details are provided in Sec. 6.2.

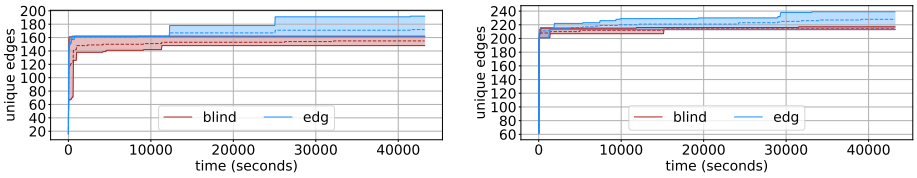
6.2 Guide Fuzzer with Instruction Trace Divergences

We select two open-source libraries, minmea [20] and libpng [21], to test whether execution divergences in these programs can guide the fuzzer. Each fuzzing run lasts 24 hours, and each scenario is repeated five times. The results for *the number of discovered unique edges over time* are shown in Figure 6.

minmea. minmea is a GPS parser that supports multiple sentence formats in the NMEA 0183 standard [22]. It is specifically designed for embedded systems because it is implemented in ISO C99 and has no dynamic memory allocation. After the fuzzing experiment, we find that our *EDG*-based fuzzing outperforms the other scenarios. *EDG* discovers 224.4 edges on average, whereas *perfect* and *blind* discover 176.6, and 155.6 edges, respectively (Figure 6a). The main reason *EDG* discovers more edges than *perfect* is that EDG generates inputs that cause segmentation faults (157 inputs). These inputs trigger system calls in the OS for crash handling, and thus the basic-block transitions within those system calls are recorded among the discovered edges.

libpng. Libpng is a widely used PNG parser library. We use Libpng version 1.6.37 provided in LibAFL's fuzzing example. The fuzzing result shows a similar outcome to minmea (Figure 6b): *EDG*-assisted fuzzing reaches the highest average number of edges (1509.6) among all scenarios. The remaining scenarios, from second high to low, are *perfect* (1473.8) and *blind* (1227.4). In addition, *EDG* also generates inputs (140) that cause segmentation faults.

In summary, we show that the fuzzer can leverage execution divergences to guide fuzzing and performs comparably to a fuzzer with full edge feedback. Whether these inputs reveal genuine bugs or exception-handling deficiencies in the test programs remains an open question for future work.



(a) Obfuscated minmea. (b) Obfuscated Lz4.

Fig. 7: Results of fuzzing real-world binaries obfuscated with indirect jumps after 12 hours. Since both binaries use indirect jumps to hide control flow, the *perfect* setting effectively reduces to the *blind* setting. Without feedback, *blind* performs worse than *EDG*, which can still guide the black-box fuzzer when static instrumentation is impossible. More details are provided in Section 6.3.

6.3 Fuzzing Indirect-Jump Binary

We investigate whether our *EDG*-based fuzzing can help the fuzzer test obfuscated binaries whose basic-block addresses are hidden through a jump table. Since the jump targets are resolved only during execution, static instrumentation cannot determine these addresses in the obfuscated binaries. As a result, the *perfect* setting effectively reduces to *blind* fuzzing. Because *simple-div* causes memory-exhaustion problems when fuzzing obfuscated binaries as well, we focus only on *EDG*, our optimized divergence-detection-based approach, in this section. Note that the number of discovered unique edges is measured from the *non-obfuscated* version of the binary. We reuse the inputs generated while fuzzing the obfuscated binaries, replay them on the non-obfuscated binaries, and trace the edges. For each obfuscated program, we run fuzzing under the *EDG* and *blind* scenarios for 12 hours and repeat each scenario three times.

Obfuscated Minmea. As Figure 7a demonstrates, *EDG* overall performs better than *blind* in terms of the number of unique edges discovered in the non-obfuscated `minmea` binary (171.67 vs 155). Neither scenario triggers any segmentation fault observed in Section 6.2.

Obfuscated lz4. lz4 [23] is a well-known compression algorithm and is widely shipped in many operating systems. After the fuzzing evaluation, *EDG* still performs better than *blind* (228 vs 214.33, Figure 7b), even though the blind fuzzer achieves 20% higher throughput. However, *EDG* in the worst case performs as poorly as the worst-case *blind* scenario: only 213 unique edges are discovered.

As a result, we conclude that *EDG* can help black-box fuzzing when static instrumentation is impossible.

7 Discussion

Anti-Fuzzing Protected Binary. Anti-Fuzzing approaches [24,25] install fake paths in a binary to mislead coverage-guided fuzzers into exploring traps, which

waste the fuzzers' time and computational resources. Since these fake paths are input-sensitive, even minor input mutations made by the fuzzers can easily trigger new edges along these fake paths. The fuzzers regard these edges as interesting and therefore spend more effort exploring the fake parts of the binary. Unfortunately, an *EDG* cannot effectively mitigate fake paths either because these fake paths can create various execution paths during runtime, which in turn create divergences that also mislead *EDG*.

Dynamic JIT Obfuscation. Some advanced obfuscation techniques [12,26] leverage JIT compilation to generate and obfuscate machine code at runtime. Under these techniques, the same function may produce different machine code each time it is invoked during execution. Because *EDG* cannot analyze the semantics of dynamically generated machine code, it treats all dynamically generated code originating from the same function as separate execution instances. As a result, *EDG* produces a large amount of false-positive feedback for the fuzzer.

EDG vs Dynamic Instrumentation. Although dynamic instrumentation tools like DynamoRIO [19] can discover the basic blocks hidden in a jump table by resolving the memory address during runtime, directly using such tools to provide edge feedback for the fuzzer cannot solve fuzzing scenarios where an obfuscated binary contains millions of bogus basic blocks that are always executed. When these bogus basic blocks are executed, DynamoRIO generates a huge number of edges as feedback, thereby overwhelming the fuzzer's edge map. In contrast, *EDG* only generates edge feedback when meaningful execution-trace differences appear. We conclude that *EDG* can assist fuzzing based on dynamic instrumentation when testing obfuscated binaries that leverage jump tables to hide basic blocks and contain an extremely large number of bogus basic blocks.

EDG with other types of execution traces. *EDG* can also be applied to non-instruction traces if the analyst has a robust model for detecting their divergences. For instance, the analyst can collect power traces from IoT devices to train a divergence-detection model and use the model together with *EDG* to generate edge feedback for guiding a fuzzer.

8 Related Work

Non-instrumented fuzzing treats the target largely as a black box and relies on input structure and externally observable states rather than compiler-inserted coverage instrumentation. Grammar-based fuzzing leverages the input structure of a target to mutate inputs without direct instrumentation. Havrikov et al. [27] propose systematic coverage of grammar productions via k-paths. This approach benefits non-instrumented fuzzing because it improves the quality and diversity of generated inputs. Olsthoorn et al. [28] combine search-based testing with grammar-based fuzzing to generate highly structured inputs. *EDG* can assist grammar fuzzers by providing additional execution-level information about a non-instrumented target.

Several prior works use externally observable states to guide fuzzing. DistFuzz [14] uses inter-node messages as feedback for black-box fuzzing without

source instrumentation. Similarly, Snipuzz [29] infers message snippets from IoT responses to guide mutation. Kim et al. [13] propose Intender, which uses intent-state transitions as external guidance for network fuzzing and exposes semantic bugs. de Ruiter et al. [15] infer TLS protocol state machines via black-box testing and inspect them for spurious transitions. RESTler [30] generates REST API request sequences from specifications to explore cloud states and infer producer-consumer dependencies. Chen et al. [31] present IoTFuzzer, which leverages protocol states between mobile apps and IoT devices. DifFuzz [32] is a resource-guided fuzzer for detecting time- and space-based side-channel vulnerabilities. Although *EDG* is not directly applicable to all of these prior scenarios, *EDG* can still be beneficial for state-guided fuzzers that test a single target entity.

9 Conclusion

In this work, we discuss approaches for collecting suitable feedback for fuzzing in scenarios where neither classic binary instrumentation nor static analysis are possible. In particular, we aim to detect the execution of novel code segments. To differentiate execution traces, we introduce the concept of execution divergence and show how it can be used to detect novel code execution, which can subsequently guide the fuzzer. We first introduce the simple divergence detection approach (Section 3.3), which directly compares execution traces to determine whether the current execution trace is unique. We then argue that, while the simple approach can be used to guide a fuzzer, it struggles with repeated execution of code segments. To address this false-positive problem, we develop algorithms such as Match First Traversal and Backlinking Candidates to reconstruct a graph from execution traces, namely the *Execution Divergence Graph* (Section 4). This EDG is essentially a CFG-like structure based on the code observed during execution so far.

We implement the approach and experimentally demonstrate that execution-divergence-based approaches are effective for fuzzing real-world programs. We show that, in many scenarios, our EDG-based approach reaches similar coverage to that of traditional full instrumentation of the target, assuming such instrumentation is possible. Our results also show that the EDG approach enables feedback in scenarios where traditional instrumentation is not possible. Even in such constrained settings, our *EDG*-based fuzzing can still discover timeouts and crashes in obfuscated binaries, whereas the black-box fuzzer finds none. Our artifact can be found at `https://anonymous.4open.science/r/EDG-BBC4`

References

1. B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of unix utilities," *Communications of the ACM*, 1990.
2. Caca Labs, "zzuf," http://caca.zoy.org/wiki/zzuf.
3. A. Helin, "RADAMSA," https://gitlab.com/akihe/radamsa.

4. Peach Tech, "Peach Fuzzer," https://peachtech.gitlab.io/peach-fuzzer-community/.
5. Google, "Technical "whitepaper" for afl-fuzz," https://raw.githubusercontent.com/google/AFL/61037103ae3722c8060ff7082994836a794f978e/docs/technical_details.txt, 2019.
6. A. Fioraldi, D. Maier, D. Zhang, and D. Balzarotti, "LibAFL: A framework to build modular and reusable fuzzers," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2022.
7. H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: Fuzzing by program transformation," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.
8. P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.
9. S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "kafl: Hardware-assisted feedback fuzzing for os kernels," in *Proceedings of the USENIX Security Symposium*, 2017.
10. C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, "REDQUEEN: fuzzing with input-to-state correspondence," in *Proceedings Network and Distributed System Security Symp. (NDSS)*, 2019.
11. C. Domas, "movfuscator," 2025. [Online]. Available: https://github.com/xoreaxeaxeax/movfuscator?tab=readme-ov-file
12. Tigress, "The tigress c obfuscator," 2025. [Online]. Available: https://tigress.wtf/
13. J. Kim, B. E. Ujcich, and D. Tian, "Intender: Fuzzing intent-based networking with intent-state transition guidance," in *Proceedings of the USENIX Security Symposium*, 2023.
14. Y. Zou *et al.*, "Blackbox fuzzing of distributed systems with multi-dimensional inputs and symmetry-based feedback pruning," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.
15. J. de Ruiter and E. Poll, "Protocol state fuzzing of tls implementations," in *Proceedings of the USENIX Security Symposium*, 2015.
16. C. Wang, "A security architecture for survivability mechanisms," Ph.D. dissertation, University of Virginia, 2001.
17. C. Linn and S. K. Debray, "Obfuscation of executable code to improve resistance to static disassembly," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2003.
18. M. Böhme, V. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2016.
19. google, "DynamoRIO," 2026. [Online]. Available: https://dynamorio.org
20. K. Moczek, P. Szymczak, and Contributors, "minmea," https://github.com/kosma/minmea, 2023.
21. Greg Roelofs and the contributers, "libpng," 2025. [Online]. Available: https://www.libpng.org/pub/png/libpng.html
22. Contributors to Wikimedia projects, "NMEA 0183 - Wikipedia," 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=NMEA_0183&oldid=1324066764
23. lz4, "lz4," 2026. [Online]. Available: https://github.com/lz4/lz4
24. E. Güler, C. Aschermann, A. Abbasi, and T. Holz, "Antifuzz: Impeding fuzzing audits of binary executables," in *Proceedings of the USENIX Security Symposium*, 2019.

25. J. Jung, H. Hu, D. Solodukhin, D. Pagan, K. H. Lee, and T. Kim, "Fuzzification: Anti-fuzzing techniques," in *Proceedings of the USENIX Security Symposium*, 2019.
26. "MyJIT," 2015. [Online]. Available: https://myjit.sourceforge.net/index.htm
27. N. Havrikov and A. Zeller, "Systematically covering input structure," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019.
28. M. Olsthoorn, A. van Deursen, and A. Panichella, "Generating highly-structured input data by combining search-based testing and grammar-based fuzzing," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE), NIER Track*, 2020.
29. X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, and Y. Xiang, "Snipuzz: Black-box fuzzing of iot firmware via message snippet inference," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.
30. V. Atlidakis, P. Godefroid, and M. Polishchuk, "Restler: Stateful rest api fuzzing," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2019.
31. J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W. C. Lau, M. Sun, R. Yang, and K. Zhang, "Iotfuzzer: Discovering memory corruptions in iot through app-based fuzzing," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.
32. S. Nilizadeh, C. S. Păsăreanu, and Y. Noller, "Diffuzz: Differential fuzzing for side-channel analysis," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2019.